With the users added to `.setUp()`, it is time to start writing the test. This view expects a logged-in user, which you do in a test by calling `.client.login()`. The following listing logs in the user that will be the owner, does a GET, and validates the resulting HTTP `200` status code.

Listing 8.6 Verify the form can be fetched using GET

```
# bands/tests.py
...
class TestBands(TestCase):
    ...
    def test_edit_venue(self):
        self.client.login(username="owner",
            password=self.PASSWORD)

        # Verify the page fetch works
        url = "/bands/edit_venue/0/"
        response = self.client.get(url)
        self.assertEqual(200, response.status_code)
```

Log in with the username owner that was created in .setUp().

Call the URL with argument 0, indicating venue creation.

Check the view returned the right HTTP status.

This code uses an argument of `0` in the URL, indicating the desire to create a new `Venue` object. As the code is only checking a `GET`, the `0` isn't strictly necessary, but using any other ID would require that `Venue` to exist or the page would `404`. Alternatively, you could create a `Venue` object first and use its ID.

After fetching the form, it is time to test adding a new venue. The `.client.post()` method takes a dictionary that maps to the form fields used in the page. For now, let's populate only the `name` and `description` fields. Listing 8.7 shows the code that tests `Venue` object creation. To determine whether object creation worked, the database gets queried for the new object; there will be only one. Next, the object's fields get checked, making sure they match what was sent in the POST. Newly created `Venue` objects also have the logged-in user assigned as one of their controllers, so this attribute needs to be validated as well.

Listing 8.7 Test creating a Venue

```
# bands/tests.py
...
class TestBands(TestCase):
    ...
    def test_edit_venue(self):
        ...
        # Create a new Venue
        data = {
            "name":"Name",
            "description":"Description",
        }
        response = self.client.post(url, data)

        self.assertEqual(302, response.status_code)
```

Dictionary with key–value pairs that correspond to the page's form

Post the data to the URL that was defined in the previous listing.

A successful creation redirects to another page, using a 302 HTTP status code for redirection.